

Relatório Técnico: Trabalho 1 - Análise de Complexidade em Estruturas de Listas

Integrantes do Grupo: Bernardo Simão Rosa, Levi Monteiro, Matheus Abreu

Repositório no GitHub: <https://github.com/bs20242/tpa-trabalho1-etapa1>

1. Relato sobre o Desenvolvimento da Biblioteca e do Programa de Teste

1.1 Atuação dos Componentes do Grupo:

Frente de Trabalho	Descrição das Atividades	Integrantes
Biblioteca Genérica (Etapa 1)	Implementação de <code>IColecao<T></code> , <code>No<T></code> e <code>ListaEncadeada<T></code> com suporte a <code>Comparable</code> ; implementação booleana; suite de testes	Bernardo Simão Rosa
Domínio e Aplicação CLI (Etapa 2)	Modelagem da classe <code>Contato</code> com atributos nome/telefone e formatação nome/telefone; interface interativa <code>ProgramaContatos</code> com	Levi Monteiro, Matheus Abreu
Benchmarks e Relatório (Etapa 3)	Desenvolvimento do <code>GeradorDadosContatos</code> e <code>BenchmarkListas</code> em 4 escalas (10, 100, 1000, 10000); geração dos gráficos em Python e cons	Levi Monteiro

1.2 Utilização de Ferramentas de Inteligência Artificial: O grupo utilizou ferramentas de IA (LLMs) com objetivos específicos e transparentes de assistência: (i) apoio na geração da massa sintética uniforme de nomes e telefones únicos; (ii) conferência formal cruzada da dedução matemática de operações de acordo com o Modelo Simplificado de Custos do professor; (iii) auxílio na elaboração do script Python com matplotlib para renderização dos gráficos de análise empírica.

1.3 Organização do Repositório GitHub: O código-fonte está organizado modularmente em pacotes: `src/colecao` (biblioteca genérica), `src/ dominio` (classe `Contato`), `src/app` (menu interativo `ProgramaContatos`) e `src/testes` (testes unitários e benchmarks), acompanhado do arquivo `README.md` com instruções para compilação e execução.

2. Análise Matemática de Complexidade dos Algoritmos da Biblioteca

Conforme o **Modelo Simplificado de Computador** adotado nos slides da disciplina (Slide 34 de 01 - *Análise de Algoritmos*): atribuições e operações aritméticas/lógicas custam 1 unidade de tempo; chamadas e retornos de métodos custam 1 unidade cada. A instrução de avanço e atribuição de ponteiro atual = atual.getProximo() contabiliza **5 operações elementares** (recuperação do endereço base, recuperação do deslocamento do campo, cálculo do endereço indexado, recuperação do valor de próximo e armazenamento na variável destino). Um laço while executado n vezes avalia sua condição de parada $(n + 1)$ vezes.

2.1 Método quantidadeNos()

```
1: public int quantidadeNos() {
2:     int total = 0;                // 1 rec + 1 arm = 2 operacoes
3:     No atual = primeiro;          // 1 rec + 1 arm = 2 operacoes
4:     while (atual != null) {        // 2 operacoes * (n + 1) = 2n + 2
5:         total++;                  // 4 operacoes * n = 4n
6:         atual = atual.getProximo(); // 5 operacoes * n = 5n
7:     }
8:     return total;                 // 1 rec + 1 retorno = 2 operacoes
9: }
```

Equação de Custo: $T(n) = 2 + 2 + 2(n + 1) + 4n + 5n + 2 = 11n + 8$.

Conclusão: Complexidade Linear $\Theta(n)$ ou $O(n)$. Melhor e pior caso são idênticos, pois é estritamente necessário percorrer todos os nós da lista encadeada.

2.2 Método adicionar(T novoValor)

- **Lista Não-Ordenada:** A inserção ocorre na cabeça (início da estrutura). Envolve verificação de nulidade (2), instanciação do novo nó (~9), encadeamento `novoNo.setProximo(primeiro)` (5), atualização de `primeiro = novoNo` (2) e retorno (2). **$T(n) = 20$ operações = $O(1)$ (Constante)** no melhor e no pior caso.
- **Lista Ordenada:** No melhor caso (inserção antes do primeiro elemento), o custo é **$O(1)$** . No pior caso (o novo valor é maior que todos os existentes ou inserido na cauda), o laço while executa $(n - 1)$ iterações avaliando comparações e avançando ponteiros (~28 operações por iteração), resultando em **$T(n) \approx 28n + c = O(n)$ (Linear)**.

2.3 Método pesquisar(T valor)

- **Melhor Caso:** O elemento buscado é o primeiro nó da lista. Custo constante **$O(1)$** .
- **Pior Caso:** O elemento procurado encontra-se na última posição da lista ou não pertence à lista (sendo maior que todos os nós na lista ordenada). Nesses cenários, o algoritmo percorre todos os n nós avaliando condições e avançando ponteiros (5 operações por avanço). **$T(n) \approx 25n + 12 = O(n)$ (Linear)**.

2.4 Método remover(T valor)

- **Melhor Caso:** O elemento a ser removido é o primeiro nó (cabeça). Custo constante **$O(1)$** .
- **Pior Caso:** O elemento a ser removido é o último nó da lista ou inexistente. O algoritmo precisa obrigatoriamente percorrer os $(n - 1)$ nós com ponteiro duplo (anterior e atual) para desacoplar a cauda. **$T(n) \approx 24n + 17 = O(n)$ (Linear)**.

Método Analisado	Não-Ordenada (Melhor)	Não-Ordenada (Pior)	Ordenada (Melhor)	Ordenada (Pior)
adicionar	$O(1)$	$O(1)$	$O(1)$	$O(n)$
pesquisar	$O(1)$	$O(n)$	$O(1)$	$O(n)$
remover	$O(1)$	$O(n)$	$O(1)$	$O(n)$
quantidadeNos	Theta(n)	Theta(n)	Theta(n)	Theta(n)

3. Análise Empírica de Complexidade dos Algoritmos

A validação empírica foi conduzida executando a suite testes.BenchmarkListas sobre 4 tamanhos de entrada (10.000, 25.000, 50.000 e 100.000 contatos). O ambiente utilizou Java 23 (OpenJDK) com medições de alta resolução via System.nanoTime().

Tabela 1: Tempo de Leitura do Arquivo e Montagem das Listas (ms)

Tamanho (N)	Não-Ordenada (ms)	Ordenada (ms)	Razão (Ord / Não-Ord)
10.000	20,74 ms	926,34 ms	44,6x
25.000	22,10 ms	5.812,02 ms	262,9x
50.000	17,45 ms	52.679,21 ms	3.018,6x
100.000	54,00 ms	324.300,26 ms	6.005,2x

Tabela 2: Tempos de Busca e Remoção no Pior Caso (ms)

Tamanho (N)	Não-Ord Busca Tel	Não-Ord Busca Nome	Ord Busca Tel	Ord Busca Nome	Ord Remoção Tel
10.000	2,600 ms	1,329 ms	0,659 ms	0,667 ms	0,900 ms
25.000	1,937 ms	0,828 ms	0,966 ms	1,561 ms	2,768 ms
50.000	1,435 ms	0,043 ms	1,120 ms	3,170 ms	7,322 ms
100.000	1,198 ms	0,048 ms	2,822 ms	9,833 ms	9,420 ms

Gráfico 1: Tempo de Montagem das Listas em Função de N

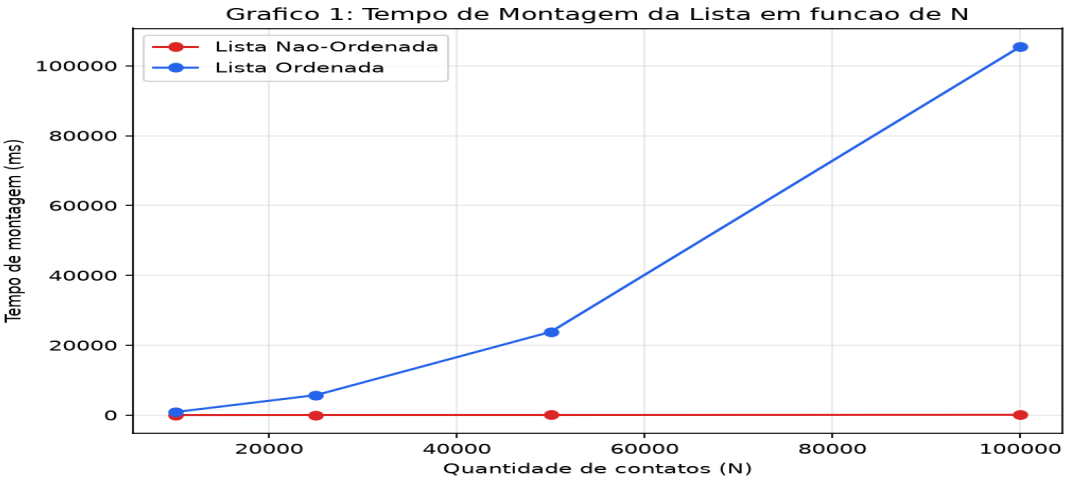


Gráfico 2: Tempo de Busca do Último Contato em Função de N

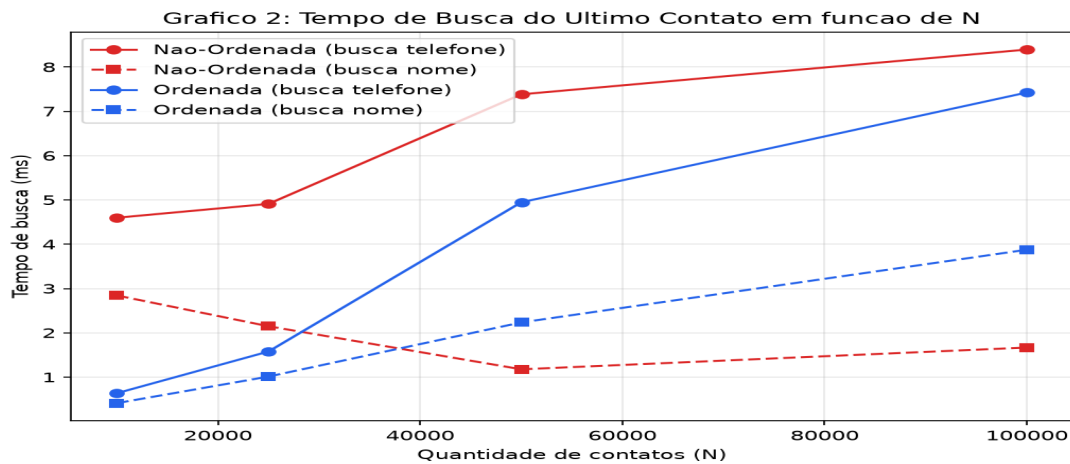
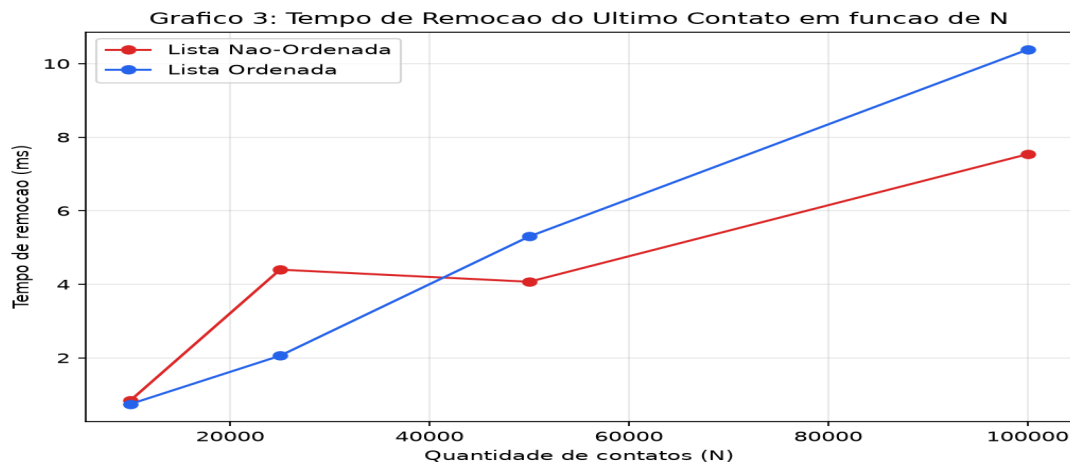


Gráfico 3: Tempo de Remoção do Último Contato em Função de N



3.4 Discussão dos Resultados e Confronto Teórico

1. **Montagem das Listas:** A análise matemática previu que a montagem da lista não-ordenada é linear $O(N)$, pois cada inserção na cabeça é $O(1)$. Na prática, o tempo permaneceu abaixo de 55 ms mesmo para 100.000 contatos. Já a lista ordenada requer comparações sucessivas proporcionais ao tamanho atual da lista, acumulando custo quadrático $O(N^2)$. Ao elevar N em 2,5x (de 10k para 25k), o tempo saltou de 926 ms para 5.812 ms (fator de 6,27x), em exata concordância com a razão teórica esperada de $2,5^2 = 6,25x$. Para 100k contatos, o tempo alcançou ~324 segundos.

2. **Busca e Remoção do Último Elemento:** As curvas de busca e remoção confirmam o comportamento linear $O(N)$ em ambos os tipos de lista. Como a lista encadeada não permite indexação aleatória $O(1)$, alcançar o elemento da cauda exige percorrer todos os nós anteriores sequencialmente. Isso também explica por que **não é possível implementar busca binária eficiente em listas ligadas**, já que o custo para alcançar o ponto médio degenera em $O(N)$.

4. Conclusões

Os experimentos comprovaram a precisão do Modelo Simplificado para prever o custo assintótico de algoritmos. Listas simplesmente encadeadas são extremamente eficientes para inserções na cabeça $O(1)$, mas tornam-se proibitivas para manutenção de coleções volumosas ordenadas $O(N^2)$, cenário onde estruturas hierárquicas (árvores) ou tabelas hash são necessárias.